

K-nearest neighbours and approximate search

Predict from the closest training examples: the distance, the choice of K , feature scale, and finding neighbours fast.

THE QUESTION

How can we predict a new point from the training examples closest to it, and find those examples quickly when there are millions?

THE INTUITION

Similar inputs tend to have similar outputs. Keep the whole training set; for a query, find the K nearest points and let them vote or average. What matters is the features and their scale, the distance, K , and how the search is done.

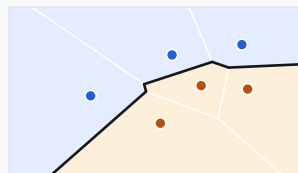
KEY TAKEAWAY

K trades variance (small K) for bias (large K): pick it on validation data, after scaling. Exact search costs $O(ND)$ per query; kd-trees and LSH scan a small part of the data and may miss a neighbour.

store X and y → scale features → query q → N distances → K smallest → vote or average

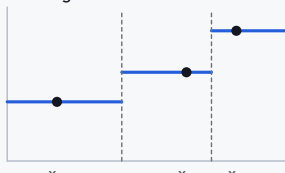
1 Similar inputs get similar outputs

1-NN classification: Voronoi cells



boundary: pieces of perpendicular bisectors

1-NN regression: a staircase



x_1 switch at midpoints x_2 x_3

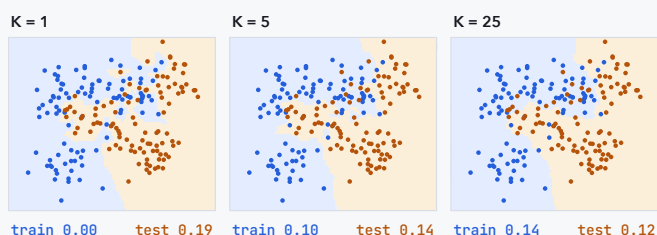
For a query q , compute the distance to every stored x_i , take the K closest and predict their majority class, or their mean for regression. With $K = 1$ each training point owns its Voronoi cell, the region closer to it than to any other point, so the boundary is piecewise linear; in 1-D the prediction steps at the midpoints $(x_i + x_{i+1})/2$.

2 KNN is lazy and non-parametric

	KNN: lazy, non-parametric	linear regression: eager
training	store the table	solve for θ
per query	N distances: $O(ND)$	one dot product: $O(D)$
memory	all $N \times D$ numbers	$D + 1$ numbers
assumes	near means similar	a linear form

Lazy: training only stores the data; the work happens per query. **Non-parametric:** what the model keeps grows with N , while a linear model keeps $D + 1$ numbers. The ideas differ: a decision tree is non-parametric but eager. Lazy suits online settings, since a new row costs nothing to add.

3 Small K follows noise, large K smooths



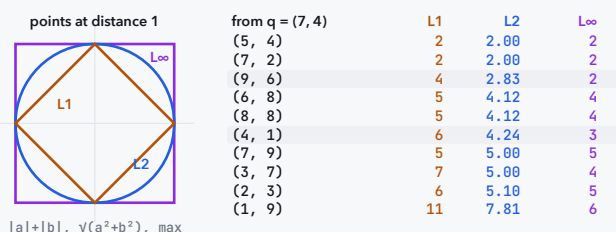
One training set of 200 points; each class mixes three Gaussian blobs (our data, seeded). $K = 1$ draws islands around single points: training error 0, test error 0.19. $K = 25$ is smoother and does best of the three, 0.117. Few neighbours: low bias, high variance (the bias and variance sheet).

4 Choose K on held-out data



Averaged over 40 training sets. Training error is 0 at $K = 1$ (each point is its own neighbour), so it cannot choose K . Test error falls from 0.169 to 0.125 at $K = 19$, then climbs to 0.325 at $K = 199$ as the vote drifts to the overall majority; the Bayes rule gets 0.113. Use odd K for two classes, and pick it by cross-validation.

5 The distance decides who counts as near



$d_p(a, b) = \left(\sum_j |a_j - b_j|^p \right)^{1/p}$: $p = 1$ Manhattan, $p = 2$ Euclidean, $p = \infty$ the largest coordinate gap; for binary features, Hamming distance counts differing bits. On the notebook's ten points, (9, 6) is third by L_2 but ties for first under L_∞ , and (4, 1) moves from sixth to fourth.

6 Scale features before measuring distance

	5-NN test accuracy	weight's share of d^2
raw units	55.0%	98.7%
min-max to [0, 1]	88.0%	37.6%
standardised	88.1%	47.8%

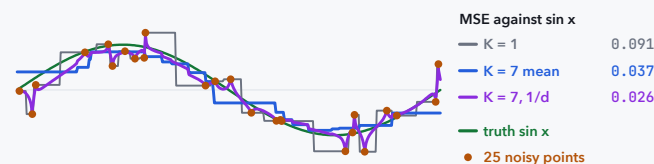
200 training fruit, 2000 test fruit, $K = 5$ (illustrative data)

Apples against oranges from redness (0 to 1, informative) and weight in grams (alike for both). Raw, weight supplies 98.7% of each squared distance and 5-NN scores 55.0%, near a coin toss. Min-max scaling or standardising, fitted on the training set, gives 88.0% and 88.1%.

Aggregation, high dimensions and fast search

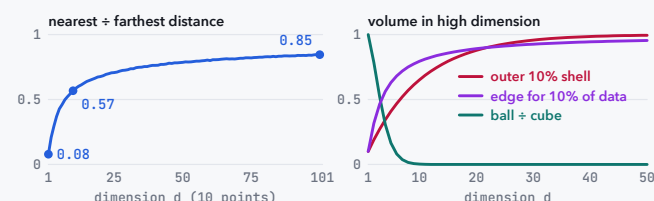
Weighted votes, distances in many dimensions, and trees and hashes that trade exactness for speed.

7 Aggregate with a mean, a vote or weights



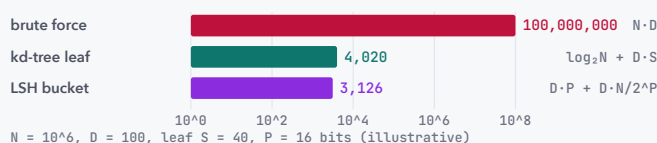
Regression averages the K targets (the median resists outliers); classification takes the mode. Weights $1/d$ let close neighbours count more: on 25 noisy samples of $\sin x$ they cut the MSE of $K = 7$ from 0.037 to 0.026. They can flip a vote: red neighbours at 0.5 and 0.8 lose 2 to 3 against blue at 1.5, 1.7, 2.0, but win 3.25 to 1.75 by weight (illustrative).

8 In high dimensions, near and far look alike



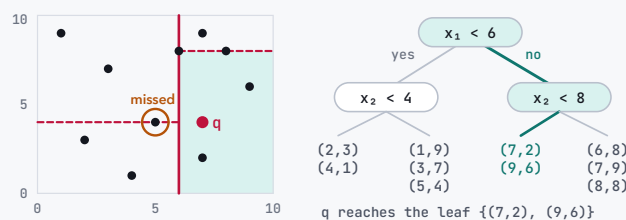
The notebook: 10 uniform points in $[0, 1]^d$, one query, 1000 runs. The nearest-to-farthest ratio rises from 0.08 at $d = 1$ to 0.57 at $d = 10$ and 0.85 at $d = 100$. Holding it near 0.08 took 10, 50, 200 and 2000 points in 1 to 4 dimensions. Right: a share $1 - 0.9^d$ of a ball lies in its outer 10% shell; a cube holding 10% of uniform data has edge $0.1^{1/d}$, 0.79 at $d = 10$, so a "neighbourhood" spans most of each axis.

9 Brute force costs $O(ND)$ per query



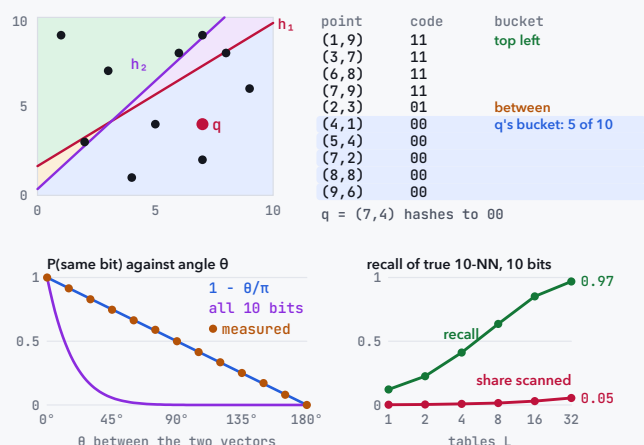
One distance takes D subtractions, D multiplications, about D additions and a square root: $O(D)$. All N cost $O(ND)$, plus a scan for the smallest. Faster search does some work at training time so that a query looks at a subset.

10 A kd-tree cuts space into boxes



Split at the median of x_1 , then of x_2 in each half, alternating: axis-aligned boxes as on the decision trees II sheet, set by the data alone. The notes' ten points give $x_1 < 6$, then $x_2 < 4$ and $x_2 < 8$. Query $(7, 4)$ reaches the leaf $\{(7, 2), (9, 6)\}$: 2 distances, not 10. It finds $(7, 2)$ at distance 2, a true nearest neighbour, but its second pick $(9, 6)$ at 2.83 misses $(5, 4)$ at 2. Build $O(N \log^2 N)$; query $O(\log N + DS)$, leaf size S .

11 LSH hashes near points to the same bucket



Draw P random hyperplanes; bit i is the side, $h_i = [w_i^T x + b_i > 0]$. The notebook's planes (seed 35) put $(7, 4)$ in bucket 00 with 5 of 10 points, both true neighbours included. A plane through the origin splits two vectors at angle θ with probability θ/π : one bit agrees with probability $1 - \theta/\pi$ (dots: measured), all P with $(1 - \theta/\pi)^P$. So use L tables: with $P = 10$ on 20,000 unit vectors in 32-D, 1 table finds 12% of the true 10 nearest scanning 0.16% of the data, 32 tables 97% scanning 5.5%. Codes cost $O(NDP)$; a query $O(DP + DN/2^P)$.

12 Pause and think

K approaches n ? Every point votes: the majority class everywhere, high bias.

Why scale features? Large units dominate the distance: 55.0% raw, 88.1% scaled.

KNN over a parametric model? Irregular boundaries, modest D , data that keeps arriving.

Naive cost of the K nearest? $O(ND)$ per query, plus a pass for the K smallest.

vote among the K nearest · scale, then tune K · distances blur as d grows · trees and hashes buy speed

Before: the unsupervised learning sheet; next: the reinforcement learning sheet. Source: lecture slides "K-nearest neighbors" and "K-nearest neighbors approximation" (handwritten notes), and the curse-of-dimensionality and KNN-variants notebooks (nipunbatra.github.io/ml-teaching). The ten points, the kd-tree and the seed-35 hyperplanes are the sources'; all other data are ours, seeded.